

CustomerGraph AI

Day-by-Day Development Plan

Auth First | Backend First | Frontend in React

Simple Project Summary

Build the backend first so all data, security, business logic, Neo4j graph, AI agents, and chatbot APIs are stable. After that, build the React frontend using real backend APIs instead of dummy UI data.

Document Detail	Value
Project Name	CustomerGraph AI
Project Type	Customer 360 Agentic AI System
Main Backend Stack	FastAPI, Neo4j, Python, JWT Auth, RBAC, LangGraph/LangChain style agents
Main Frontend Stack	React, React Router, Axios/Fetch, dashboard components, chatbot UI
Planning Style	Backend complete first, then React frontend integration
Security Rule	Auth, role-based access control, protected APIs, and audit logs from the beginning
Final MVP	Customer profile, health score, risk detection, next best action, support priority, upsell/renewal, dashboard, chatbot

1. Build Order

Important rule: Do not start React screens first. First create secure backend APIs and graph data. React should consume real APIs after backend logic is ready.

Order	Layer	What gets completed first
1	Auth and Security	Login, password hashing, JWT, role-based permissions, protected APIs, audit logs
2	Neo4j Graph Backend	Customer, contact, ticket, invoice, product, contract, usage, employee relationships
3	Business APIs	Customer profile, tickets, invoices, renewals, usage, manager dashboard metrics
4	Business Intelligence Logic	Health score, risk reason, support priority, renewal priority, upsell opportunity
5	Agentic AI Backend	Summary agent, risk agent, sales agent, support agent, renewal agent, next best action agent
6	Chatbot Backend	Graph-based question answering using safe Cypher/templates and explainable answers
7	React Frontend	Login, protected layout, dashboards, profile pages, risk pages, chatbot UI
8	Final QA and Demo	Security tests, API tests, UI tests, portfolio video/demo documentation

2. MVP User Roles and Permissions

Role	Can Access	Purpose
Admin	All modules, user management, roles, audit logs	System owner and project demo control
Sales Executive	Customer profiles, sales opportunities, renewal insights, chatbot	Prepare for calls and identify upsell or renewal actions
Account Manager	Customer profiles, health score, risk, renewals, next best actions	Protect customer relationship and prevent churn
Support Agent	Customer profile, tickets, support priority, chatbot	Prioritize important tickets with full context
Customer Success Manager	Customer health, risk reasons, usage, renewal recommendations	Track customer value and engagement
Sales Manager / Leadership	Manager dashboard, revenue at risk, risky accounts, opportunity summary	See portfolio-level customer risk and business impact

3. Suggested Backend Modules

Backend Module	Main Responsibility
auth	Register first admin, login, refresh token, logout, password hashing, JWT creation
users	Users, roles, permissions, role checks, profile endpoint
audit	Log login, failed login, protected actions, AI recommendation viewed, recommendation approved/rejected
graph	Neo4j connection, constraints, indexes, Cypher helpers, data loading
customers	Customer list, customer profile, contacts, owner, products, health summary
support	Tickets, severity, open issues, repeated complaints, support priority
billing	Invoices, delayed payments, contract value, payment risk
usage	Product usage, usage drop, usage growth, feature adoption
renewals	Renewal date, renewal risk, renewal follow-up actions
intelligence	Health score, risk detection, next best action, upsell logic
agents	Customer summary, risk, support priority, sales opportunity, renewal, manager insight agents
chatbot	Natural language questions converted into safe graph queries and explainable answers

4. 40-Day Backend-First and React-Second Roadmap

How to follow this plan				
Days 1-26 are backend work. Do not build final React UI during this time. Days 27-38 are React frontend work using real backend APIs. Days 39-40 are final testing, fixing, documentation, and demo preparation.				
Day	Area	Task	Output	Check
Day 1	Backend - Planning + Auth	Freeze MVP scope, define user roles, decide permissions, define protected modules and access rules.	MVP scope, roles table, permission matrix.	You know who can access each feature.
Day 2	Backend - Project Setup	Create backend folder structure, virtual environment, requirements, config file, .env pattern, logging setup.	FastAPI backend runs locally.	GET /health returns OK.
Day 3	Backend - Auth Design	Create user model, password hashing plan, JWT access token, refresh token, login flow, role table design.	Auth design and database schema ready.	Auth data model is clear before coding APIs.
Day 4	Backend - Auth APIs	Implement login, first admin creation, token generation, refresh token, logout placeholder.	Auth APIs available.	Valid user can login and get token.
Day 5	Backend - RBAC	Implement protected route dependency, role checker, permission checker, and role-specific access.	Protected APIs working.	Unauthorized user is blocked.
Day 6	Backend - Audit Logs	Add audit log table/model and middleware/helper for login, failed login, and important user actions.	Audit log backend ready.	Login and failed login events are recorded.
Day 7	Backend - Neo4j Setup	Install Neo4j local/Aura setup, configure driver, create connection service and health check.	Neo4j connected to backend.	Backend can verify Neo4j connection.
Day 8	Backend - Graph Schema	Design graph nodes and relationships: Customer, Contact, Product, Ticket, Invoice, Contract, Employee, Usage, Meeting, Feedback.	Graph schema document and Cypher constraints.	Neo4j schema matches BRD requirements.
Day 9	Backend - Sample Data	Create sample CSV/JSON data for customers, contacts, products, tickets, invoices, contracts, usage, meetings, feedback.	Demo data files ready.	Each customer has enough data for risk and recommendation logic.
Day 10	Backend - Data Loader Part 1	Build loader for customers, contacts, employees, products, account ownership, and purchased products.	Customer base graph created.	Neo4j Browser shows customer-product-owner graph.
Day 11	Backend - Data Loader Part 2	Load support tickets, invoices, contracts, renewals, usage data, meetings, feedback into Neo4j.	Full sample customer graph ready.	One customer profile has connected tickets, invoice, usage, renewal.
Day 12	Backend - Customer APIs	Create customer list API, customer detail API, customer search API, and customer graph summary API.	Customer APIs ready.	API returns Customer 360 profile data.
Day 13	Backend - Operational APIs	Create APIs for tickets, invoices, usage, renewal, products, contacts, and account owner details.	Business data APIs ready.	All profile page sections can be powered by backend.
Day 14	Backend - Health Score	Create customer health score rules using product usage, open tickets, ticket severity, payment delay, renewal date, feedback.	Health score service and API.	Each customer returns score, level, and reason.
Day 15	Backend - Risk Detection	Create risk detection logic for churn risk, support risk, payment risk, low engagement, upcoming renewal risk.	Risk API and risk reasons.	High-risk customer list is generated from real graph data.
Day 16	Backend - Next Best Action Rules	Create rule-based next best action engine combining risk, usage, support, invoice, renewal, and customer value.	Next best action API.	Each recommendation has action, reason, priority, confidence.
Day 17	Backend - Renewal + Upsell Logic	Build renewal follow-up logic and sales opportunity logic based on plan usage, usage growth, product gap, renewal date.	Renewal API and upsell API.	System identifies customers ready for follow-up or upgrade.
Day 18	Backend - Support Priority Logic	Create support priority engine using ticket severity, customer value, renewal impact, repeated complaints, and SLA age.	Support priority API.	Support tickets are ranked with business reason.
Day 19	Backend - Manager Dashboard APIs	Create aggregated APIs: total customers, high-risk customers, revenue at risk, open critical tickets, delayed invoices, renewals, opportunities.	Manager metrics APIs ready.	Dashboard can show real summary numbers.
Day 20	Backend - Agent Framework Setup	Create agents folder, shared agent state, tool interface, graph query tool, business logic tool, and response format.	Agent framework skeleton.	Agents can call backend services safely.
Day 21	Backend - Customer Summary	Build summary agent that	Customer Summary Agent.	API returns meeting-ready

	Agent	creates simple customer brief from profile, tickets, usage, invoice, renewal, and meetings.		summary.
Day 22	Backend - Risk + Support Agents	Build Risk Detection Agent and Support Priority Agent. Output must include answer, reason, data evidence, and confidence.	Risk Agent and Support Agent.	Agents explain decisions instead of only giving labels.
Day 23	Backend - Sales + Renewal Agents	Build Sales Opportunity Agent and Renewal Agent using graph data and rules.	Sales Agent and Renewal Agent.	Agents identify upsell and renewal follow-up opportunities.
Day 24	Backend - Next Best Action Agent	Build aggregator agent that combines risk, support, payment, usage, renewal, and sales signals into one best action.	Next Best Action Agent.	Action output is explainable and human-review friendly.
Day 25	Backend - Chatbot API	Create chatbot endpoint. Use safe predefined Cypher templates or controlled query generation. Add source data in answer.	Graph chatbot backend.	User can ask customer questions and receive data-grounded answers.
Day 26	Backend - Backend QA Freeze	Test all backend APIs, auth, RBAC, audit logs, Neo4j queries, risk logic, agents, chatbot. Create Postman collection.	Backend MVP complete.	React can now start using real APIs.
Day 27	Frontend React - Setup	Create React project, folder structure, environment config, API client, router, base layout, shared components.	React app runs locally.	React can call backend /health.
Day 28	Frontend React - Login	Build login page, token storage, auth context, logout, protected routes, invalid-token handling.	Login UI working.	User cannot access dashboard without login.
Day 29	Frontend React - Layout + RBAC Menu	Build sidebar, top bar, role-based menu, page shell, error boundary, loading components.	Protected dashboard shell.	Different roles see correct menu options.
Day 30	Frontend React - Main Dashboard	Build dashboard cards for total customers, high-risk customers, renewals, delayed invoices, open critical tickets, revenue at risk.	Main dashboard page.	Cards show backend data, not dummy data.
Day 31	Frontend React - Customer List	Build customer table with search, filters, risk badge, owner, health score, renewal date, and open tickets count.	Customer list page.	User can open customer profile from list.
Day 32	Frontend React - Customer 360 Profile	Build customer profile page with overview, products, contacts, tickets, invoices, usage, renewal, health score, AI summary.	Customer detail page.	Business user sees complete customer context.
Day 33	Frontend React - Risk Page	Build high-risk customer page with risk level, reasons, owner, renewal date, revenue impact, recommended action.	Risk customers page.	Risk reasons are visible and easy to understand.
Day 34	Frontend React - Support Page	Build support priority page with tickets ranked by urgency, customer value, renewal impact, and escalation reason.	Support intelligence page.	Support agent knows which ticket to handle first.
Day 35	Frontend React - Sales + Renewal Pages	Build sales opportunity page and renewal follow-up page with suggested product, confidence, reason, and next step.	Sales and renewal pages.	Sales team can identify upsell and renewal actions.
Day 36	Frontend React - AI Recommendation UI	Create reusable components for AI summary, next best action, risk reasons, confidence, and approve/reject action.	AI insight components.	Recommendations are shown with reasons and human review.
Day 37	Frontend React - Chatbot UI	Build chatbot page for customer questions. Show answer, reason, source customer data, and suggested follow-up questions.	AI chatbot UI.	User can ask questions from frontend.
Day 38	Frontend React - Integration Polish	Connect all pages to backend APIs, handle loading, empty states, auth expiry, API errors, and role-based redirects.	Integrated React MVP.	No final page depends on dummy data.
Day 39	Final QA - End-to-End	Run end-to-end tests: login, roles, customer profile, risk page, next action, support priority, chatbot, audit logs.	Bug-fixed MVP.	Full demo flow works from login to chatbot.
Day 40	Final Delivery - Portfolio Ready	Prepare README, architecture diagram, API list, screenshots, demo script, sample credentials, and deployment notes.	Portfolio-ready project package.	Project is ready to show in interview or portfolio.

5. Backend Completion Checklist Before React Starts

- Auth works: login, token, logout behavior, protected route dependency.
- RBAC works: Admin, Sales, Account Manager, Support Agent, Customer Success Manager, Manager roles have different access.
- Audit logs work: login, failed login, AI recommendation viewed, action approved or rejected.
- Neo4j graph has demo customer data with products, tickets, invoices, usage, contracts, renewals, contacts, employees.
- Customer Profile API returns one complete Customer 360 response.
- Health score API returns score, level, and clear reasons.
- Risk API returns risk level, risk reasons, priority, and revenue impact where available.
- Next Best Action API returns action, reason, confidence, and human-review status.
- Agent APIs return grounded answers using customer data.
- Chatbot API answers graph-based customer questions with reason and source context.
- Postman collection or API test script is ready for frontend integration.

6. Frontend React Pages to Build After Backend

React Page	Backend APIs Needed	What user should see
Login Page	/auth/login, /auth/me	Secure login before dashboard access
Main Dashboard	/dashboard/summary	Customer counts, high-risk customers, renewals, delayed invoices, revenue at risk
Customer List	/customers	Searchable customer table with health and risk badges
Customer 360 Profile	/customers/{id}, /customers/{id}/summary	Full profile, tickets, invoices, usage, renewal, AI summary, next best action
Risk Customers	/risk/customers	Risk level, risk reason, owner, renewal date, recommended action
Support Priority	/support/priority	Ranked tickets with escalation reason
Sales Opportunities	/sales/opportunities	Upsell/cross-sell suggestions with confidence
Renewals	/renewals/upcoming	Upcoming renewal follow-ups and risk reason
AI Chatbot	/chatbot/query	Natural language question answering from Neo4j graph data
Audit/Admin	/admin/users, /audit/logs	Users, roles, and important system activity

7. Backend API Priority List

Priority	API Group	Example Endpoints
P0	Auth	POST /auth/login, POST /auth/refresh, GET /auth/me, POST /auth/logout
P0	Customers	GET /customers, GET /customers/{id}, GET /customers/{id}/graph-summary
P0	Dashboard	GET /dashboard/summary, GET /dashboard/revenue-risk
P1	Health and Risk	GET /customers/{id}/health, GET /risk/customers, GET /customers/{id}/risk
P1	Next Best Action	GET /customers/{id}/next-best-action, POST /actions/{id}/approve
P1	Support	GET /support/priority, GET /customers/{id}/tickets
P1	Sales/Renewal	GET /sales/opportunities, GET /renewals/upcoming
P2	Agents	GET /agents/customer-summary/{id}, GET /agents/risk/{id}, GET /agents/manager-summary
P2	Chatbot	POST /chatbot/query
P2	Admin/Audit	GET /admin/users, POST /admin/users, GET /audit/logs

8. Final Demo Flow

- Login as Admin or Manager.
- Open Main Dashboard and show total customers, high-risk customers, renewals, delayed invoices, and revenue at risk.
- Open Customer List and select one risky customer.
- Open Customer 360 Profile and show tickets, invoice status, usage drop, renewal date, health score, and AI summary.

- Show why the customer is high risk with exact reasons.
- Show Next Best Action with reason and confidence.
- Open Support Priority page and show which ticket should be handled first.
- Open Sales Opportunity page and show upsell recommendation.
- Ask chatbot: Why is this customer high risk? Then ask: Which customers have renewal in the next 30 days?
- Show audit logs proving user actions and AI recommendation views are tracked.

9. Simple Rule to Remember

Development rule

Backend first means: no final React dashboard until backend APIs, auth, Neo4j data, risk logic, agents, and chatbot backend are working. React should only display real backend intelligence.

Prepared for CustomerGraph AI project planning. Date: 30 May 2026.